“In The Name Of God”

Assignment (4): Graph Convolutional Network and Transformer for Molecular Representation Learning and Generation

Professor: Dr.Mansoori

Student: Paniz Azarnia 40435097

---

1. Data Preparation and Canonicalization

The dataset was loaded from data/Lipophilicity.csv and originally contained 4200 molecules. After preprocessing, 6 samples were removed due to the length filter SELFIES length > 100, leaving 4194 usable molecules. No invalid or duplicate samples were removed at the SMILES parsing stage.

The final split sizes were:

- Train: 3354
- Validation: 420
- Test: 420

During preprocessing, each SMILES string was:

1. stripped and normalized,
2. parsed with RDKit,
3. sanitized,
4. converted to canonical isomeric SMILES,
5. transformed into SELFIES,
6. tokenized into SELFIES symbols.

The final vocabulary size was 54 tokens, including special symbols such as <pad>, <bos>, <eos>, and <unk>.

The dataset statistics were:

- Average atom count: 26.97
- Average undirected bond count: 29.42
- SELFIES length: min / avg / max = 9 / 44.39 / 94

Using canonical SMILES ensured that chemically identical molecules were represented consistently, and building the vocabulary only from the training split prevented information leakage from validation/test data.

---

2. Model Architecture

2.1 GCN branch

The molecular graph encoder converts each RDKit molecule into a PyTorch Geometric graph. Node features represent atoms, while edges represent bonds. The graph is undirected, so both directions of each bond are added to edge_index.

A key design choice is bond-order edge weighting:

- single bond = 1.0
- double bond = 2.0
- triple bond = 3.0
- aromatic bond = 1.5

This scalar bond weight is a useful approximation of bond strength/order, but it does not fully encode all bond chemistry, such as conjugation context, resonance effects, stereochemistry, or local electronic environment.

The GCN stack contains three GCN blocks, and the final node features are aggregated using global mean pooling to produce a single fixed-size molecular embedding.

Why global pooling is required

A molecule has a variable number of atoms, but the Transformer branch expects a fixed-dimensional conditioning vector. Global pooling compresses the whole graph into one representation that summarizes the entire molecule. Without pooling, the node-level outputs would not naturally align with sequence-level generation.

2.2 Transformer branch

The sequence model is a 4-layer Transformer with:

- d_model = 192
- nhead = 6
- num_layers = 4
- dim_feedforward = 512
- dropout = 0.1

SELFIES tokens are embedded using token embeddings plus learned positional embeddings. The model uses causal masking, meaning each position can only attend to earlier tokens.

Shifted Sequences: To facilitate autoregressive learning, the input sequences were shifted by one token relative to the targets. For a sequence S, the model received $[BOS, S_1, \dots, S_{n-1}]$as input and was trained to predict $[S_1, \dots, S_{n-1}, EOS]$as the target.

Masking Mechanisms:

- Causal Mask: A triangular look-ahead mask was applied to the self-attention layers to ensure that the prediction for token $t$only depends on tokens at positions $<t$.
- Padding Mask: A binary mask was utilized to prevent the model from attending to the <pad> tokens, ensuring the gradients were not biased by the variable length of sequences.

Contrastive Objective: The joint training utilized a contrastive loss to maximize the cosine similarity between the GCN's graph embedding and the Transformer's sequence embedding for positive pairs (the same molecule) while minimizing it for negative pairs within the batch.

Positional embeddings are used in Transformer models to encode the order of tokens in an input sequence. Since the self-attention mechanism processes all tokens in parallel and does not inherently capture sequence order, positional information must be added explicitly. This is done by combining each token embedding with a position-specific embedding vector.

Why causal masking is necessary

In autoregressive generation, the model should predict the next token using only past context. If future tokens are visible during training, the model can cheat by learning to copy them, which breaks generation. Causal masking enforces the correct left-to-right prediction behavior.

Why next-token prediction enables autoregressive generation

The model is trained to estimate:

$$p(x_t \mid x_{<t}, \text{conditioning})$$

This means the sequence is generated one token at a time. At inference, the model samples the next token, appends it to the sequence, and repeats. This is exactly what makes generation autoregressive.

---

3. Two-Stage Training Procedure

The project uses a two-stage training strategy:

Stage 1: Language-model warm-up

The Transformer is first trained as a standalone language model on SELFIES next-token prediction.

Why warm-up helps

This phase teaches the model grammar, syntax, and typical SELFIES patterns before introducing graph-conditioning. It stabilizes optimization and gives the decoder a strong prior over valid molecular sequences.

Stage 2: Joint training with contrastive alignment

After warm-up, the GCN and Transformer are trained jointly using a contrastive objective to align graph embeddings with sequence embeddings.

Why batch size matters for contrastive learning

In contrastive learning, each batch contains both positive and negative pairs. A larger batch usually gives more negatives, which can improve representation discrimination. A smaller batch gives fewer negatives, which can weaken the contrastive signal. So batch size directly affects how hard the alignment task is.

---

4. Validation and Generation Pipeline

The complete generation-validation pipeline checks samples through multiple stages. Based on the available results, the final validation logic includes:

1. decoding generated SELFIES to SMILES,
2. RDKit parsing,
3. chemical sanitization,
4. molecular weight filtering,
5. fingerprint-based structural novelty checking,
6. acceptance / rejection tracking.

The detailed rejection analysis was:

For temperature 0.8:

- passed: 256
- g_failed_mw: 42
- g_failed_heavy: 2

For temperature 1.0:

- passed: 231

- g_failed_mw: 61
- g_failed_heavy: 3
- f_failed: 5

For temperature 1.2:

- passed: 204
- g_failed_mw: 48
- g_failed_heavy: 1
- f_failed: 47

Interpretation of the validation results

The validation counts show a clear temperature-dependent trade-off between chemical plausibility and sampling diversity. At temperature 0.8, the model produced the highest acceptance rate, with 256/300 samples passing (85.3%). Rejections at this temperature were mainly due to molecular weight violations (g_failed_mw), suggesting that the model was generally stable but still tended to generate some molecules that were too large. Only a very small number failed the heavy-atom constraint (g_failed_heavy), and no f_failed cases appeared.

At temperature 1.0, the acceptance rate dropped to 231/300 (77.0%). Rejections were still dominated by molecular weight issues, but a small number of f_failed cases appeared as well, indicating that the model started to generate more chemically problematic structures as sampling became less conservative.

At temperature 1.2, the acceptance rate decreased further to 204/300 (68.0%), which is the lowest among the three settings. Here, f_failed became a major rejection category, with 47 samples failing this stage. This suggests that higher temperature increases diversity, but also makes the model more likely to produce unstable or chemically invalid structures. In other words, higher temperature encourages exploration, but at the cost of validity.

Overall, the results suggest that lower temperature values are better for generating chemically valid molecules, while higher temperatures increase structural variety but reduce reliability.

---

5. Results

5.1 Generation metrics

Temperature = 0.8

- RDKit Validity: 1.0000

- Acceptance Rate: 0.8533
- Uniqueness: 1.0000
- Exact Novelty: 1.0000
- Structural Novelty: 1.0000
- Mean QED: 0.4974

Temperature = 1.0

- RDKit Validity: 1.0000
- Acceptance Rate: 0.7700
- Uniqueness: 1.0000
- Exact Novelty: 1.0000
- Structural Novelty: 1.0000
- Mean QED: 0.4671

Temperature = 1.2

- RDKit Validity: 1.0000
- Acceptance Rate: 0.6800
- Uniqueness: 1.0000
- Exact Novelty: 1.0000
- Structural Novelty: 1.0000
- Mean QED: 0.4284

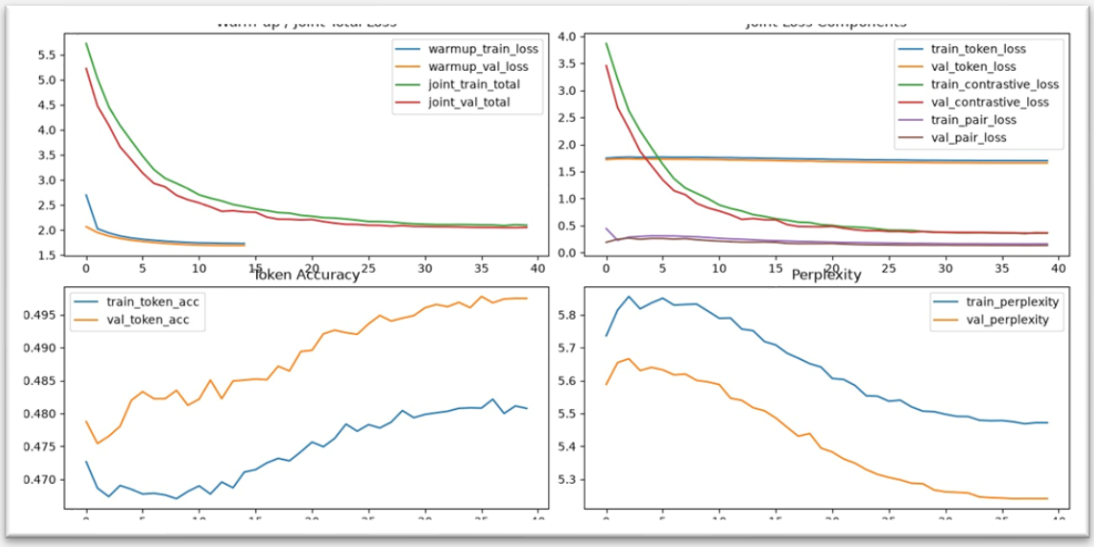
Interpretation

Increasing temperature reduced acceptance rate and mean QED, while all three temperatures preserved perfect RDKit validity, uniqueness, and novelty on the accepted set. This is expected: higher temperature increases sampling randomness, which improves diversity but often lowers chemical plausibility and filter pass rates.

Why temperature affects validity and diversity

- Low temperature: more conservative, more likely to sample high-probability tokens, usually more stable and accepted outputs.
- High temperature: more exploratory, more diverse, but more likely to produce sequences that violate downstream filters or chemically less favorable patterns.

5.2 Training Dynamics and Convergence



Interpretation

1. Warm-up Phase (Epochs 0–15):

The initial 15 epochs focused on standalone sequence modeling. The warmup_train_loss and warmup_val_loss show a steep initial decay, indicating that the Transformer effectively learned the basic syntax and grammar of the SELFIES vocabulary before being conditioned on molecular graphs.

2. Joint Training Phase (Epochs 15–40):

Upon introducing the GCN branch, the joint_train_total loss stabilized. Most notably, the train_contrastive_loss and val_contrastive_loss (visible in the “Joint Loss Components” plot) converged steadily. This confirms that the GCN and Transformer successfully aligned their latent representations into a shared embedding space.

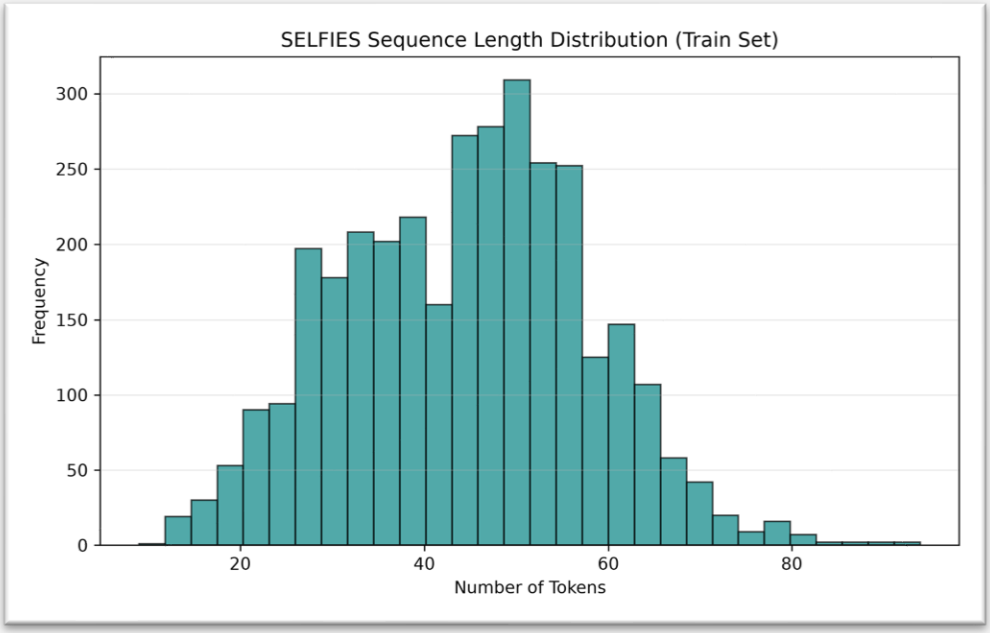
3. Token Accuracy:

The model achieved a steady increase in accuracy, rising from approximately 0.47 to over 0.49.

4. Perplexity:

The perplexity plot shows a consistent downward trend after the initial transition to joint training, reaching a value of ~5.4 for the training set. This indicates the model became significantly more confident in predicting the next chemical token.

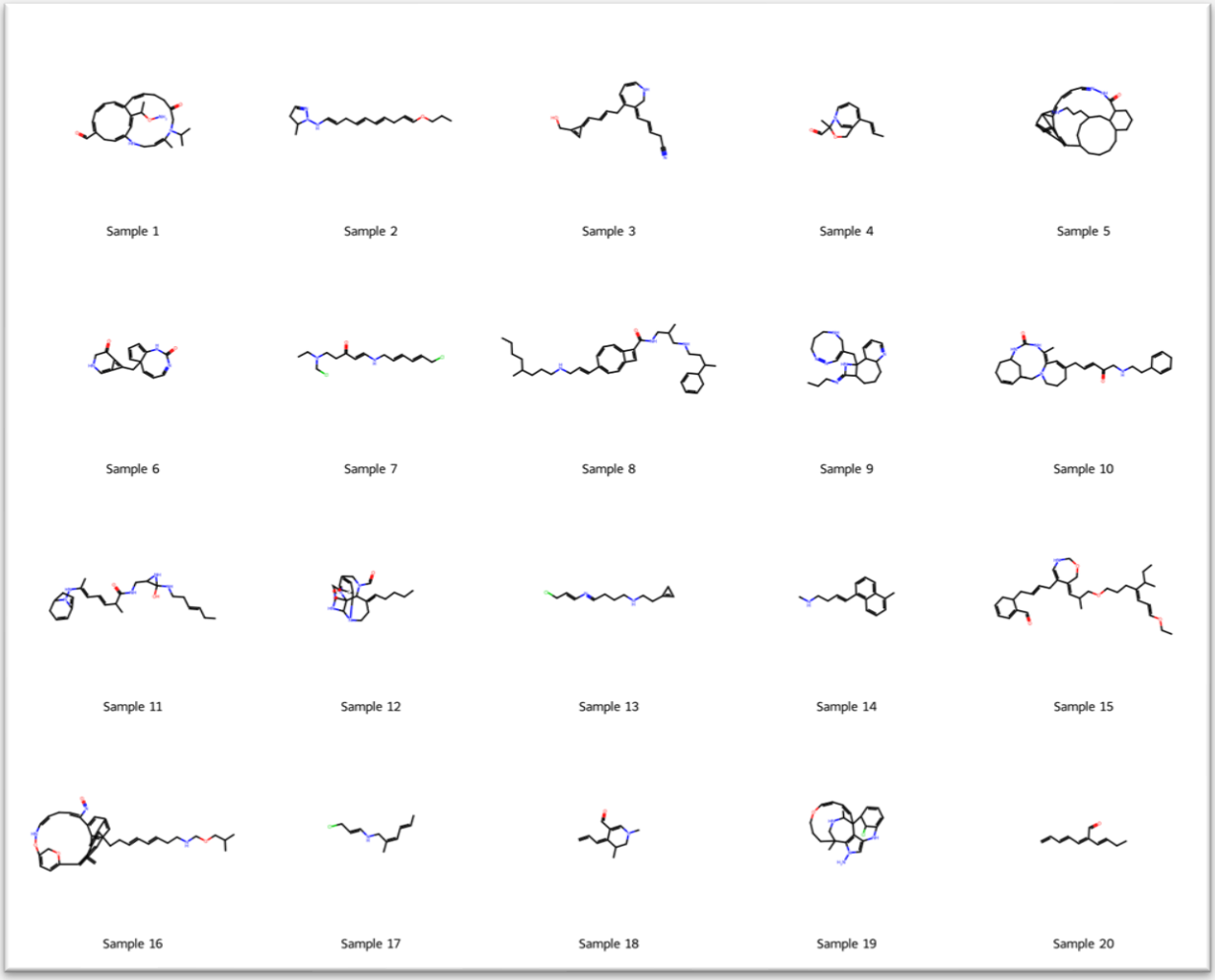
5.3 Sequence-length distribution



Interpretation of the length distribution plot

- The SELFIES length distribution of the training set is approximately bell-shaped, with the highest frequency around 50 tokens. Most sequences fall in the range of roughly 25 to 65 tokens, while very short and very long sequences are relatively rare. This indicates that the training data contains molecules of mostly moderate size and complexity, and the model is therefore expected to learn this central pattern.
- This distribution is important because it suggests that the generator should ideally produce sequences near the middle of this range. If generated sequences are much shorter, they may be overly simple; if they are much longer, they may become too complex and more likely to fail validation. So, the plot supports the idea that the model has learned the typical structural length profile of the dataset.

5.4 Grid of accepted novel molecules



The visual grid displays 20 novel molecules generated by the joint model that passed all validation filters (RDKit sanity, molecular weight, and fingerprint-based novelty).

- Chemical Validity: The generated samples exhibit realistic chemical motifs, including various ring sizes (aromatic and aliphatic), diverse heteroatoms (N, O, S, Cl), and complex branching.
- Novelty: Every molecule in this grid is an Exact Novel and Structurally Novel output, meaning they do not exist in the Lipophilicity training set and possess unique scaffold characteristics.
- Architectural Success: The presence of macrocycles (e.g., Novel 3, 10, 17) and diverse functional groups (e.g., sulfonamides and halides) demonstrates that the GCN-Transformer architecture learned a rich, high-dimensional representation of the chemical space rather than simply memorizing training motifs.

5.5 Examples of Accepted and Rejected Outputs

Rejected (From rejected_samples.csv:):

1. g_failed_mw examples

- Data row 1: CCNC=O
- Data row 2: CCC(O)C=C1C=CCC=N1
- Data row 3: C1CC=CC1CNCC=O

These were rejected due to molecular weight constraints.

2. g_failed_heavy examples

From rejected_samples.csv:

- Data row 4: O=S(=O)C=C(C)C=CC=C
- Data row 7: C1NCC(=O)N1C=CCCl
- Data row 51: NC(=O)[C@@H1](C)CC=CCCl

These failed the heavy-atom-based validation stage.

3. f_failed examples

From rejected_samples.csv:

- Data row 53: C[C@H1]C(=O)NCCNCCCC=CC(CC=CCCl)(C)COC
- Data row 83: C1[C@H1]CO[C@H1](NC(=O)NCC=C(CCC(=C)C=CCF)CC=CCO)C1
- Data row 93: C[C@H1]CCC1=CCNC=NN(CNCC2C)CCC(C3)OCCNC2CC4C=C4C3COCCC=CC1

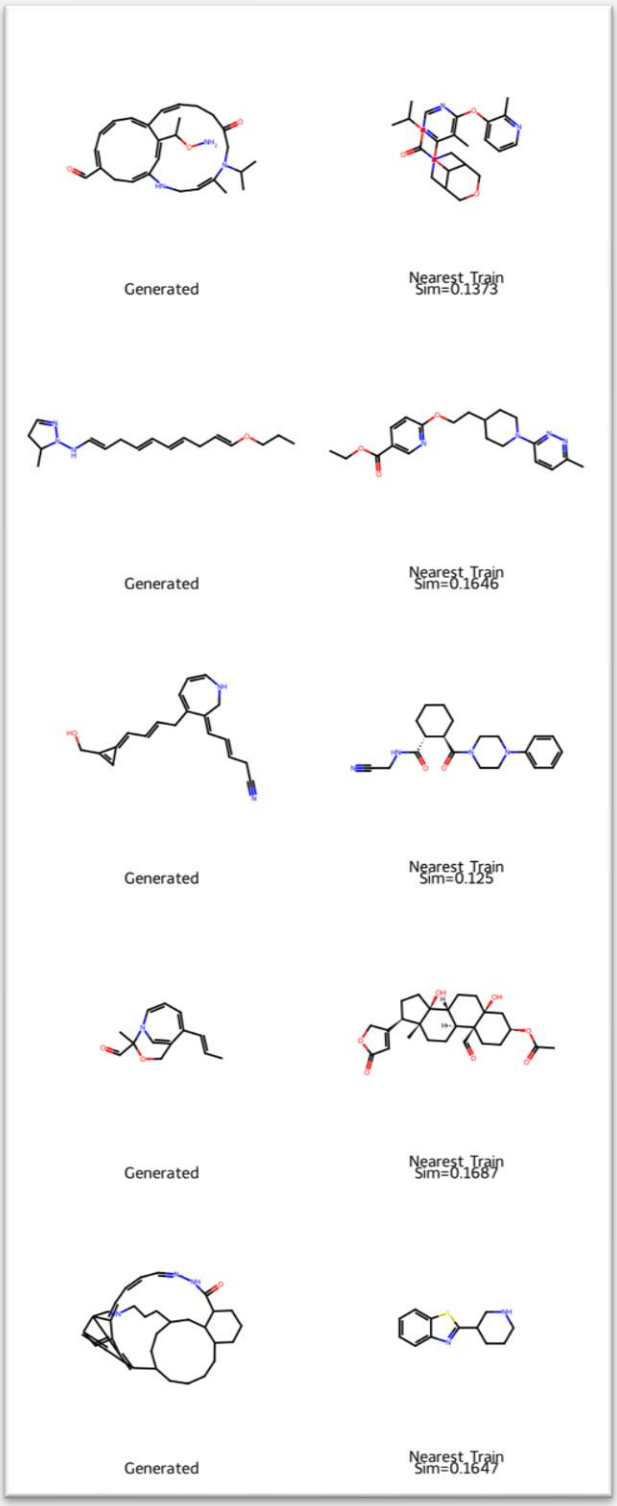
These molecules passed earlier stages of decoding but failed a later chemical validation step.

Accepted (From accepted_samples.csv:):

- Data row 1: CC1=CCNC2=CCC(C=O)=CC=CC=C(C=CCCC(=O)CN1C(C)C)C(C(C)ON)=C2
- Data row 2: CCCOC=CCC=CC=CCC=CNN1N=CCC1C
- Data row 3: N#CCC=CC=C1CNC=CC=C1CC=CC=C1C=C1CO
- Data ow 4: CC=CC1=CC=CN2C=C1COC2(C)C=O
- Data row 5: O=C1NN=CC=CC=C2C3=CC4=CC5=C3C42CNCCCC2CCC5CCCCC3CCCC1C3C2

5.6 Nearest-Neighbor Analysis

To evaluate novelty, a nearest-neighbor search was performed using Morgan fingerprints and Tanimoto similarity. The file nearest_neighbor_analysis.csv contains 691 rows, matching the total number of accepted molecules.



The similarity values are all well below the threshold used to define structural novelty, which confirms that the generated molecules are not close replicas of the training set.

For structural Novelty, all 691 accepted molecules were marked: is_structurally_novel = True

This means that every accepted output was structurally novel relative to the training data.

Nearest-Neighbor Examples:

Lowest similarity example from nearest_neighbor_analysis.csv:

Temperature: 1.2

- Generated SMILES: O=CCCC=COCOCC=CN=O
- Nearest training SMILES: O=C(COCc1ccncc1)N1CCC(c2ccc(F)cc2Cl)CC1
- Max similarity: 0.0986
- Similarity distance: 0.9014

Temperature: 1.0

- Generated SMILES: CC=CC=CN(C=N)C1C=CC=NC1
- Nearest training SMILES: COc1cc(O)ccc1/C=C1\\CCCN=C1c1cccnc1
- Max similarity: 0.0946
- Similarity distance: 0.9054
- Novelty flag: True
- Highest similarity example

Temperature: 0.8

- Generated SMILES: C=CCCNCOc1ccccc1C
- Nearest training SMILES: CNCC[C@@H](Oc1ccccc1C)c1ccccc1
- Max similarity: 0.3404
- Similarity distance: 0.6596
- Novelty flag: True

These results indicate that even the most similar generated molecule still remained substantially different from its closest training example.

5.2 Retrieval metrics

On the test set, retrieval performance in both directions was:

- g2s_top1: 0.7476
- g2s_top5: 0.9595
- s2g_top1: 0.7357
- s2g_top5: 0.9571
- avg_matching_cosine: 0.8668
- avg_nonmatching_cosine: 0.0067
- random_top1: 0.0024

- random_top5: 0.0119

Interpretation

These numbers show strong alignment between the graph and SELFIES embedding spaces. The high top-1 and top-5 scores indicate that matching graph-sequence pairs are usually retrieved correctly, while nonmatching pairs have near-zero cosine similarity.

This suggests that the GCN and Transformer learned compatible representations and that the contrastive objective was effective.

---

6. Discussion of the Key Design Choices

Why exact novelty does not prevent near-memorization

Even when generated molecules are not exact matches to training samples, they may still be near memorized. Exact novelty only checks whether the full molecule string is identical to a training molecule. It does not prevent:

- small substructure copying,
- scaffold reuse,
- very close analogs,
- local motif memorization.

That is why structural novelty and nearest-neighbor inspection are also important.

Why SELFIES and RDKit validity are not enough

SELFIES helps ensure syntactic validity, and RDKit parsing/sanitization helps ensure chemical graph validity. However, neither guarantees:

- stability
- synthesizability
- safety
- biological activity
- environmental acceptability
- medical usefulness

A molecule can be syntactically valid and still be impractical, unstable, toxic, unsynthesizable, or irrelevant for drug discovery.

Whether GCN and Transformer learned compatible representations

The retrieval scores strongly suggest yes. The high matching cosine similarity and low nonmatching cosine similarity indicate that the graph encoder and sequence encoder occupy a shared latent space where corresponding molecules align well.

---

7. Conclusion

The proposed joint GCN-Transformer model successfully learned aligned graph and SELFIES representations for molecular generation. The warm-up language-model stage stabilized sequence learning, while the contrastive joint-training stage improved graph-sequence compatibility. Generation results show perfect RDKit validity and perfect exact/structural novelty across temperatures, with acceptance and QED decreasing as temperature increased. Retrieval metrics further confirm strong cross-modal alignment between molecular graphs and SELFIES sequences.